ShareArb: Evictor-Side Responsibility for Shared Linux Page-Cache Arbitration

Anonymous Authors
Institution Withheld for Review

March 23, 2026

Abstract

Linux page-cache control for multi-tenant file-backed workloads is difficult when tenants overlap on the same pages: proportional occupancy accounting says who currently uses memory, but not who caused downstream harm when a shared page was evicted. We study a narrow response to that gap, ShareArb, a replay-side controller that augments a pCache-Account-style private/shared accounting baseline with evictor-side responsibility debt. For each shared page, ShareArb tracks a lightweight shared-responsibility vector over recent consumers and charges debt to the tenant whose miss evicts a page that other tenants are likely to reuse soon. The executed artifact reports 180 primary replay runs, 120 ablation runs, 6 oracle runs, 24 auxiliary replay perturbation runs, and 5 smoke probes under a shortened replay-only regime. Across 36 matched overlap-heavy workload-budget-seed tuples, ShareArb improves paired worst-tenant slowdown relative to pCache-Account by -1.08 (95% bootstrap CI $[-2.06, -0.23]$) and fairness by $+0.0065$ (95% CI $[0.00009, 0.0141]$). The clearest gain appears on SQLiteTraceMix-2T at medium budget, where worst-tenant slowdown falls from 17.82 to 4.59 and throughput proxy rises from 0.01737 to 0.08264. The evidence remains mixed: shared regret is inconclusive in the paired comparison against pCache-Account, ShareArb-NoDebt is competitive or better in the targeted ablations, the pre-registered support criterion is not met, and the planned live proxy study was infeasible. The artifact therefore supports ShareArb as a promising replay-side scaffold for responsibility-aware arbitration, not as decisive validation that eviction debt itself is the winning mechanism.

1 Introduction

Linux page-cache research no longer lacks mechanism. FetchBPF, PageFlex, and cache_ext show that Linux paging and file-cache policy are now programmable enough to host specialized policies [Cao et al., 2024, Yelam et al., 2025, Zussman et al., 2025]. Likewise, broad claims about tenant-aware accounting are already occupied: *pCache* separates private and shared page-cache accounting for containers [Wang et al., 2024], while Per-VM Page Cache Partitioning studies utility-driven share control with tenant-level policy choice [Sharma et al., 2016].

The narrower unresolved problem is eviction responsibility under shared residency. When multiple tenants reuse the same file-backed page, occupancy accounting can say who currently consumes cache space, but it cannot attribute who caused harm when a fault by tenant i displaced a shared page that tenant j was about to reuse. Under proportional charging, a tenant can evict a page that is only fractionally charged to it, yet most of the refault cost is paid by another tenant.

This paper studies whether a deliberately small responsibility signal helps. ShareArb starts from a pCache-Account-style private/shared baseline and adds one new quantity: decayed debt charged to the tenant whose miss evicts a shared page with predicted near-term reuse by others. The goal is not to propose a new kernel mechanism or a new occupancy theory, but to test whether a replay-side responsibility signal changes arbitration quality on overlap-heavy file-backed workloads.

The empirical outcome is directionally positive but not fully confirmatory. Across the 36 matched overlap-heavy workload-budget-seed tuples used for the main paired comparison, ShareArb improves worst-tenant slowdown relative to pCache-Account by -1.08 (95% bootstrap CI $[-2.06, -0.23]$, Wilcoxon $p = 0.077$), improves throughput proxy by $+0.00875$ (95% CI $[0.00375, 0.01459]$, $p = 0.059$), and improves fairness by

+0.00652 (95% CI $[0.00009, 0.01410]$, $p = 0.0073$). But the debt mechanism itself is not cleanly validated: against ShareArb-NoDebt, the paired slowdown delta is +0.91 (95% CI $[0.14, 1.89]$, $p = 0.214$), while the fairness and throughput intervals both span zero. The artifact's own top-level claim assessment therefore marks the pre-registered support criterion as unsupported.

Our contributions are:

- We formulate a narrow shared-page arbitration question around *evictor-side responsibility*: whether charging predicted cross-tenant harm at eviction time adds useful signal beyond proportional private/shared occupancy accounting.
- We instantiate that question in ShareArb, a conservative replay-side controller that combines proportional occupancy, a shared-responsibility vector, decayed eviction debt, and a coarse share-plus-policy action space.
- We report a replay-first artifact with 180 primary runs, 120 ablations, 6 oracle runs, 24 auxiliary replay perturbation runs, and 5 smoke probes, showing strong gains in some overlap-heavy regimes but only mixed evidence that debt itself is the decisive mechanism.

Section 2 reviews adjacent work. Section 3 describes the arbitration model and controller. Section 4 presents the replay setup, main results, ablations, and auxiliary checks. Section 5 discusses limitations and claim boundaries. Appendix A maps artifact files to the paper.

2 Related Work

Shared and weighted page-cache accounting. Wang et al. [2024] show that Linux container accounting should distinguish private from shared page-cache occupancy and proportionally charge shared pages. ShareArb assumes that style of accounting as a baseline rather than competing with it. Weight-aware page-cache control pursues fairness through occupancy and residency proportionality [Park et al., 2020], but does not explicitly assign downstream eviction harm to the evictor.

Tenant-aware page-cache control. Per-VM Page Cache Partitioning combines utility estimation with per-VM page-cache control [Sharma et al., 2016]. P2Cache similarly exposes application-directed file-cache control [Lee et al., 2023]. These systems show that online share and policy control are not novel by themselves. Here the controller is intentionally small and serves only as a harness for testing the added responsibility signal.

Programmable Linux paging mechanisms. FetchBPF, PageFlex, and cache_ext make Linux paging policy programmable through eBPF or user-space delegation [Cao et al., 2024, Yelam et al., 2025, Zussman et al., 2025]. They matter here because they remove "the kernel cannot express this" as the primary objection. They do not define the specific objective studied here: charging the tenant that triggered a harmful shared-page eviction.

Isolation, cache QoS, and shared cached objects. Shared-Page Management identifies cross-domain interference from shared-page displacement and proposes conservation-style protection for real-time reservations [Kim and Rajkumar, 2012]. ROBUS studies fair allocation when one cached object benefits many tenants at once [Kunjir et al., 2017]. Hardware-cache partitioning work likewise shows that the optimized signal matters as much as the replacement rule [Qureshi and Patt, 2006, Kim et al., 2004]. Together these papers broaden the positioning: the core issue is not Linux page-cache programmability alone, but how to define a fairness or utility signal when one cached object can benefit several principals at once. Unlike those systems, ShareArb focuses on online file-page eviction in Linux-style replay, where responsibility is inferred from recent reuse and charged at eviction time.

Trace-based cache analysis. Miniature Simulations, SHARDS, and CounterStacks establish the credibility of compact trace replay and miss-ratio estimation for storage and cache studies [Waldspurger et al., 2017, 2015, Tian et al., 2015]. Our artifact follows that methodology: the evidence is replay-first, and the paper does not claim a kernel realization.

3 Method

3.1 Problem setup

We model a set of tenants $\mathcal{T}$ issuing references to file-backed pages. Time is partitioned into control epochs of 2,000 references per tenant. Each resident page g maintains a recent-consumer set C_g over a 4,096-reference window. Pages with $|C_g| = 1$ are private; pages with $|C_g| > 1$ are shared.

The controller chooses two things at each epoch:

1. a coarse cache-share allocation on a 10-quantum simplex with at least one quantum per tenant; and
2. one replacement profile per tenant from $\{\mathrm{LRU}, \mathrm{SCAN}, \mathrm{FREQ}\}$.

The question is whether the controller should react only to occupancy, or also to the harm a tenant imposes on others when it evicts a shared page.

3.2 Baseline private and shared occupancy

ShareArb keeps a conservative pCache-Account-style baseline. For a resident page g and tenant i,

$$\mathrm{occ}_g(i) = \begin{cases} 1 & \text{if } |C_g| = 1 \text{ and } i \in C_g, \\ \frac{1}{|C_g|} & \text{if } |C_g| > 1 \text{ and } i \in C_g, \\ 0 & \text{otherwise.} \end{cases}$$

Tenant occupancy is then $\mathrm{occ}_i = \sum_g \mathrm{occ}_g(i)$. This baseline answers "who uses cache space now?" but not "who caused the next miss?"

3.3 Shared-responsibility vector

For each shared page g, ShareArb tracks a shared-responsibility vector

$$r_g(i) = \frac{\mathrm{reuse}_g(i)}{\sum_{j \in C_g} \mathrm{reuse}_g(j)},$$

where $\mathrm{reuse}_g(i)$ is a decayed reuse counter with $\alpha = 0.875$ computed over recent references. The vector sums to one and degenerates to a single entry for private pages. Intuitively, $r_g(i)$ estimates which tenants are most likely to benefit if g remains resident.

3.4 Evictor-side responsibility debt

When tenant i faults and the chosen victim is a shared page g, ShareArb charges the evictor for predicted cross-tenant harm:

$$\Delta\mathrm{debt}_i(g) = \sum_{j \neq i} r_g(j)\, \mathrm{cost}_j(g).$$

The main matrix uses a family-constant miss-cost proxy, where $\mathrm{cost}_j(g)$ is the calibrated miss-gap for the workload family. Debt decays once per control epoch:

$$\mathrm{debt}_i(t+1) = \lambda\, \mathrm{debt}_i(t) + \mathrm{new_harm}_i(t),$$

with default half-life equal to one working-set turnover.

This separates three signals that occupancy-only schemes conflate:

- occupancy: who currently holds cache space;
- benefit: who is likely to reuse the page soon; and
- responsibility: who triggered the eviction that destroyed that benefit.

3.5 Controller

Each tenant receives a pressure score

$$\text{pressure}_i = \max(0, \text{occ}_i - \text{share}_i) + \eta\, \text{debt}_i,$$

with default $\eta = 0.5$ when debt is enabled. The controller enumerates all legal share allocations and policy assignments and scores them with the same fixed objective for every method. The objective combines estimated utility, pressure, allocation variance, slowdown penalty, and a small policy-specific bonus for scan- or frequency-friendly phases. The controller is intentionally small; the paper is not about advanced optimization.

Reduction rule. Before each epoch update, ShareArb disables debt by setting $\eta = 0$ if the previous epoch's shared-page fraction is below 0.05 or if SRV confidence is too weak (mean normalized entropy above 0.95 with fewer than two observed reuses per shared page). This makes the method falsifiable: when overlap disappears, it should collapse toward occupancy-only arbitration.

4 Experiments

4.1 Setup

Artifact scope and executed protocol. The artifact is a CPU-only replay engine and controller simulator, not a kernel implementation. The original plan called for 300 non-live replay runs (180 primary plus 120 ablations), 6 oracle runs, and 12 live runs. Execution diverged in two material ways. First, the smoke probes triggered the runtime fallback: synthetic traces were shortened from the planned 80,000 to 20,000 references per tenant, and the SQLite traces from 60,000 to 15,000. Second, the live step was skipped because writable cgroup v2 `memory.high` controls plus reliable cache-reset and residency-sampling hooks were unavailable in the unprivileged environment. The executed artifact therefore reports 180 primary runs, 120 ablations, 6 oracle runs, 24 auxiliary replay perturbation runs, and 5 smoke probes, with claims explicitly scoped to replay-only evidence.

Workloads and budgets. The primary matrix contains five workload families: OverlapShift-2T, ScanVsLoop-2T, SQLiteTraceMix-2T, SQLiteTraceMix-3T, and DisjointPhase-2T. In the executed seed-11 traces, realized overlap ratios are 0.186, 0.263, 0.406, 0.517, and 0.000, respectively. Medium-budget capacities from `calibration/manifest.json` are 1,497, 1,422, 98, 175, and 1,756 pages for those families. The corresponding family-constant miss penalties are 194.011, 173.8335, 171.792, 168.936, and 173.545 μs.

Methods. We compare PrivateOnly-Utility, pCache-Account, pCache-Account+Policy, and ShareArb in the primary matrix. Ablations use ShareArb-NoDebt, ShareArb-UnitCost, Uniform-SRV, ShareArb-HalfLife0.5, ShareArb-HalfLife2.0, and NoReduction. OracleOverlap is evaluated on OverlapShift-2T and SQLiteTraceMix-2T at medium budget.

Metrics. The primary metric is worst-tenant slowdown. For tenant i, slowdown is average replay latency divided by the isolated average latency from the calibration manifest; run-level slowdown is $\max_i \text{slowdown}_i$. Aggregate throughput proxy is total references divided by total replay latency. Jain fairness is computed over inverse slowdowns. Shared regret counts a lost near-term reuse when one tenant evicts a shared page and another tenant rereferences it within the next 2,048 references; the reported normalized value is `shared_regret_raw` $\times 10{,}000/|\text{trace}|$. Controller stability is reported as action changes per 10,000 references.

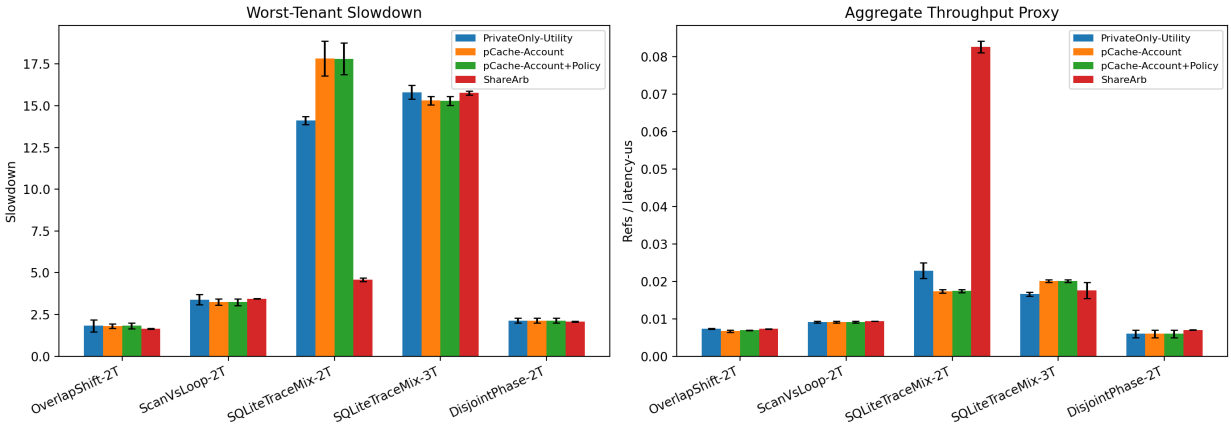

Figure 1: Medium-budget main results from the primary replay matrix. Each bar is the mean over $n = 3$ seeds for one workload family and method. Error bars are the upper bootstrap 95% half-width produced by `exp/shared/analysis.py`. The strongest gain for ShareArb appears on SQLiteTraceMix-2T, while the disjoint control remains close on slowdown but not equivalent under the pre-registered throughput criterion.

Runtime envelope. Summing emitted per-run runtimes gives 10,136.29 s (2.82 h) for the 180 primary runs, 6,533.35 s (1.81 h) for the 120 ablations, 282.49 s (0.08 h) for the 6 oracle runs, 1,055.03 s (0.29 h) for the 24 auxiliary replay checks, and 255.64 s (0.07 h) for the 5 smoke probes. Figure 6 visualizes only the primary, ablation, and oracle stages because that is what `exp/shared/analysis.py` renders; those three stages sum to 16,952.13 s (4.71 h). Adding the auxiliary replay checks brings the reported replay total to 18,007.16 s (5.00 h). Peak resident memory over all reported runs is 82.51 MB.

4.2 Main results

The main comparison against pCache-Account is positive but should be described conservatively. Across the 36 matched overlap-heavy workload-budget-seed tuples, ShareArb improves paired worst-tenant slowdown by -1.08 with a 95% bootstrap CI of $[-2.06, -0.23]$ and paired throughput proxy by $+0.00875$ with CI $[0.00375, 0.01459]$. Fairness also improves by $+0.00652$ with CI $[0.00009, 0.01410]$ and Wilcoxon $p = 0.0073$. Shared regret remains inconclusive at -0.64 with CI $[-4.38, 3.24]$ and $p = 0.424$. With only three seeds per condition, we therefore interpret the slowdown, throughput, and fairness gains as directional replay evidence, not as a broad deployment-level confirmation.

The clearest success case is SQLiteTraceMix-2T. At medium budget, ShareArb reduces worst-tenant slowdown from 17.82 under pCache-Account to 4.59, increases throughput proxy from 0.01737 to 0.08264, and lowers shared regret from 21.17 to 6.58 per 10,000 references. OverlapShift-2T shows a smaller slowdown and throughput gain at medium budget, but not a regret gain. ScanVsLoop-2T is mixed: ShareArb improves throughput and regret at medium budget, but its worst-tenant slowdown is slightly worse than pCache-Account (3.44 versus 3.25). SQLiteTraceMix-3T improves fairness at medium budget (0.847 to 0.935) but does not improve slowdown or throughput.

The disjoint control is directionally benign in several cells but does *not* satisfy the pre-registered reduction criterion. At medium budget, pCache-Account yields worst-tenant slowdown 2.13 and throughput proxy 0.00598, while ShareArb yields 2.07 and 0.00703. However, the artifact-level hypothesis check in `results.json` reports a disjoint slowdown delta of 1.97% but a disjoint throughput delta of 24.68%, exceeding the 3% tolerance. The top-level hypothesis is therefore marked unsupported even though the disjoint runs are not worse in the ordinary sense.

4.3 Budget sensitivity and mechanism behavior

Figure 2 shows that the gains are regime-specific rather than uniform. SQLiteTraceMix-2T remains the clearest success case at medium and loose budgets. OverlapShift-2T improves in slowdown at all budgets

Table 1: Exact medium-budget results from the primary replay matrix. Values are means over the three reporting seeds. Best slowdown and regret are in **bold**; best throughput is in **bold**. Lower is better for slowdown and regret, higher is better for throughput.

Family	Method	Worst slowdown ↓	Throughput proxy ↑	Shared regret / 10k ↓
OverlapShift-2T	PrivateOnly-Utility	1.822	**0.00734**	59.75
OverlapShift-2T	pCache-Account	1.799	0.00673	**37.06**
OverlapShift-2T	pCache-Account+Policy	1.818	0.00693	45.82
OverlapShift-2T	ShareArb	**1.641**	0.00728	61.25
ScanVsLoop-2T	PrivateOnly-Utility	3.388	0.00911	102.25
ScanVsLoop-2T	pCache-Account	3.247	0.00912	61.29
ScanVsLoop-2T	pCache-Account+Policy	**3.240**	0.00915	62.65
ScanVsLoop-2T	ShareArb	3.444	**0.00936**	**54.81**
SQLiteTraceMix-2T	PrivateOnly-Utility	14.11	0.02286	24.92
SQLiteTraceMix-2T	pCache-Account	17.82	0.01737	21.17
SQLiteTraceMix-2T	pCache-Account+Policy	17.81	0.01743	21.83
SQLiteTraceMix-2T	ShareArb	**4.59**	**0.08264**	**6.58**
SQLiteTraceMix-3T	PrivateOnly-Utility	15.80	0.01661	**2.17**
SQLiteTraceMix-3T	pCache-Account	15.31	0.02008	5.00
SQLiteTraceMix-3T	pCache-Account+Policy	**15.29**	**0.02010**	5.00
SQLiteTraceMix-3T	ShareArb	15.77	0.01754	7.56
DisjointPhase-2T	PrivateOnly-Utility	2.134	0.00598	50.74
DisjointPhase-2T	pCache-Account	2.134	0.00598	50.74
DisjointPhase-2T	pCache-Account+Policy	2.132	0.00598	50.19
DisjointPhase-2T	ShareArb	**2.069**	**0.00703**	**43.75**

Table 2: Ablation means on overlap-heavy families at medium budget. Values are means over the four overlap-heavy families and three seeds ($n = 12$ matched tuples per method, except UniformSRV which is only defined at medium budget). These values isolate the incremental role of debt and reuse weighting while keeping the controller scaffold fixed.

Method	Slowdown ↓	Throughput ↑	Jain ↑	Regret ↓
ShareArb	6.36	0.02920	0.9724	32.55
ShareArb-NoDebt	**6.10**	0.02805	**0.9827**	**29.23**
ShareArb-UnitCost	10.84	0.01272	0.9836	29.85
UniformSRV	6.32	**0.02996**	0.9375	30.34

but worsens regret at medium and loose budget. ScanVsLoop-2T improves throughput and some regret values but not worst-tenant slowdown at medium budget. SQLiteTraceMix-3T remains inconsistent, especially on slowdown.

Figure 3 examines whether charged debt tracks realized downstream harm. Across all ShareArb tenant-epoch rows emitted by the artifact ($n = 1{,}024$), the global correlation is positive (Pearson 0.445, Spearman 0.433). That trend is not strong enough to validate the mechanism on its own. The claim assessment records only 7 positive family-budget pairs out of the 12 overlap-heavy combinations considered in the planned threshold, which is why the overall hypothesis is marked unsupported in `results.json`.

4.4 Ablations

The ablation study is the main reason we stop short of a stronger claim. Against ShareArb-NoDebt, the paired slowdown delta for full ShareArb is $+0.91$ with a 95% bootstrap CI of $[0.14, 1.89]$; the throughput delta is $+0.00065$ with CI $[-0.00041, 0.00209]$; the fairness delta is -0.00241 with CI $[-0.00818, 0.00247]$; and the regret delta is $+2.24$ with CI $[-1.80, 6.36]$. In other words, the matched evidence does not show a clear benefit from enabling debt, and the slowdown comparison points in the opposite direction.

Table 2 summarizes medium-budget overlap-heavy means. Full ShareArb averages 6.36 worst-tenant slowdown, while ShareArb-NoDebt averages 6.10. UniformSRV closely matches the slowdown level and slightly exceeds throughput, but with substantially worse fairness. ShareArb-UnitCost is unstable, driven largely by the poor SQLiteTraceMix-2T medium-budget outcome.

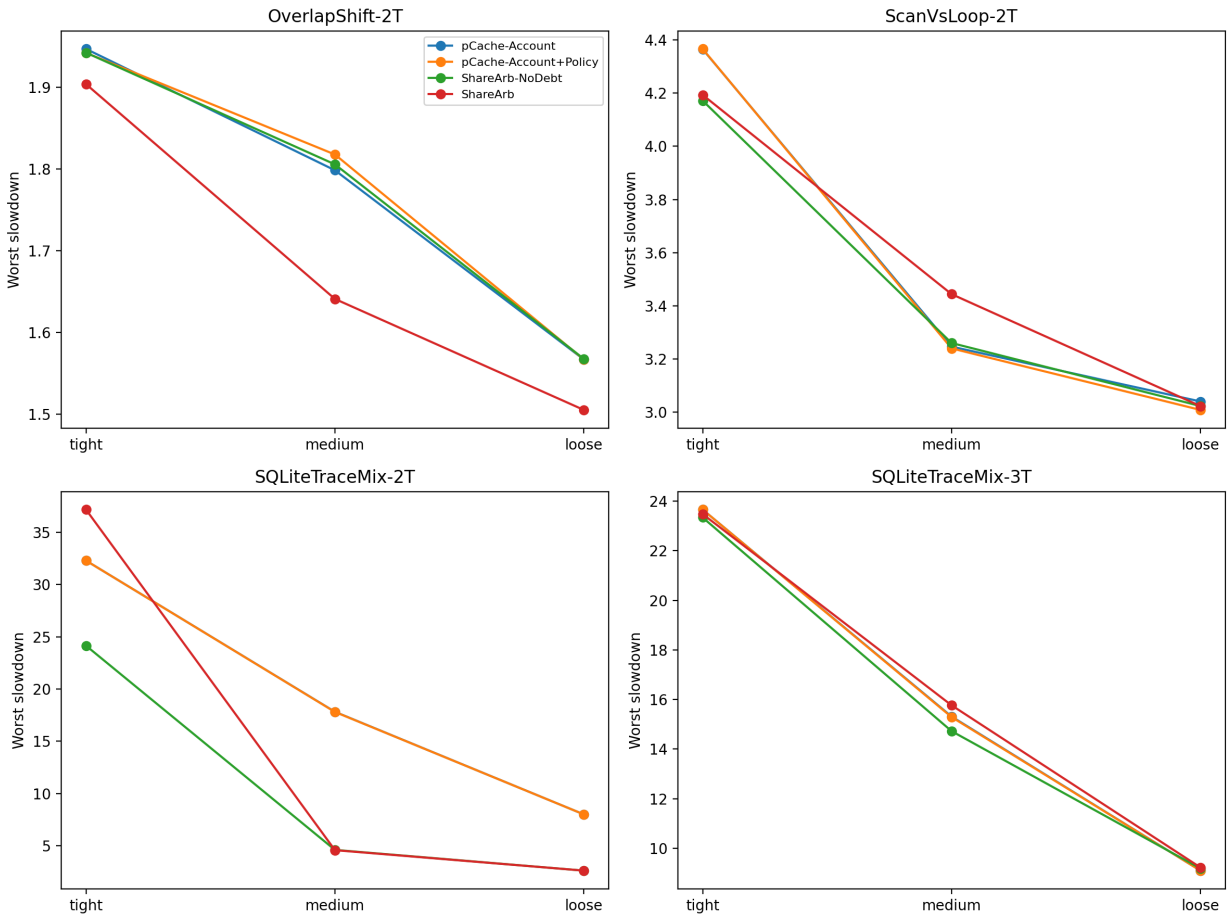

Figure 2: Budget sensitivity from the primary replay matrix. Each line shows mean worst-tenant slowdown over $n = 3$ seeds for one family-method pair at tight, medium, and loose budgets. Benefits of ShareArb are concentrated in SQLiteTraceMix-2T rather than spread uniformly across all overlap-heavy families.

4.5 Oracle and auxiliary replay validation

OracleOverlap is intentionally limited to two medium-budget families and should be read only as an upper reference. Across its six runs, mean worst-tenant slowdown is 9.85, mean throughput proxy is 0.01268, and mean shared regret is 24.46 per 10,000 references. It is therefore useful as a bounded reference point, not as a uniform upper envelope for every reported metric.

The auxiliary perturbation check compares pCache-Account and ShareArb on raw and burst-perturbed captured SQLite streams. ShareArb wins 9 of the 12 replay conditions on worst-tenant slowdown. We report these only as replay-side robustness checks. They do not replace the skipped live validation step and are not used as evidence for deployment readiness.

5 Discussion and Limitations

The main conclusion is narrower than the original hypothesis. ShareArb is useful in some overlap-heavy replay regimes, especially SQLiteTraceMix-2T, but the artifact does not justify a broader claim that evictor-side debt is independently responsible for the improvement. It also does not satisfy the full pre-registered support criterion because the positive debt-alignment count misses threshold and the disjoint throughput tolerance check fails.

Three limitations dominate.

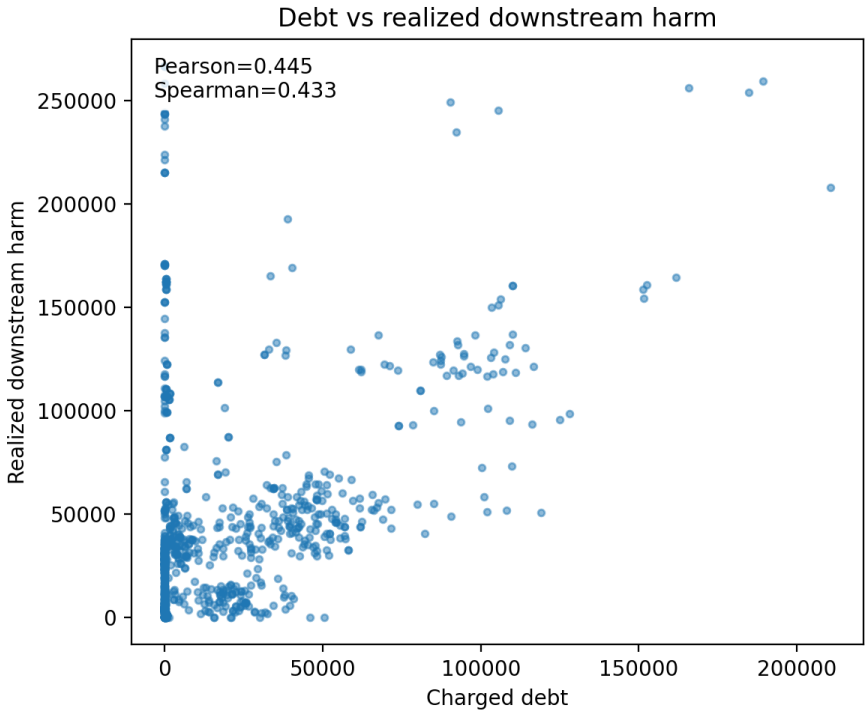

Figure 3: Charged debt versus realized downstream harm, one point per tenant-epoch for all ShareArb epoch rows produced by the artifact ($n = 1{,}024$). The overall trend is positive, but the spread remains wide, so this figure supports only a weak alignment claim.

Debt is not cleanly validated. The no-debt ablation is competitive and sometimes better. The paired slowdown comparison against ShareArb-NoDebt is unfavorable to full ShareArb, and the fairness, throughput, and regret intervals all span zero.

Replay-only scope. The planned live proxy study was skipped because the required cgroup and residency-observation controls were unavailable in the unprivileged environment. The paper therefore makes only replay-scoped claims. The 24 auxiliary SQLite perturbation runs are helpful robustness checks, but they are still replay.

Shortened horizons and regime dependence. The reported synthetic runs use 20,000 references per tenant and the SQLite traces use 15,000. The evidence therefore concerns bounded replay horizons rather than long steady-state executions. It also remains regime-specific: the strongest benefits appear in a two-tenant trace-derived overlap case, while SQLiteTraceMix-3T does not improve worst-tenant slowdown at medium budget.

6 Conclusion

This paper asked a narrow question: does page-cache arbitration improve if the system charges tenants not only for occupancy, but also for the harm they cause when evicting shared pages? ShareArb implements that idea with a shared-responsibility vector, decayed eviction debt, and a conservative share-plus-policy controller layered over pCache-Account-style occupancy accounting.

The answer is partial. Relative to pCache-Account on the 36 matched overlap-heavy workload-budget-seed tuples, ShareArb improves paired worst-tenant slowdown by -1.08 and fairness by $+0.00652$, with the clearest workload-level gain on SQLiteTraceMix-2T at medium budget. At the same time, shared regret

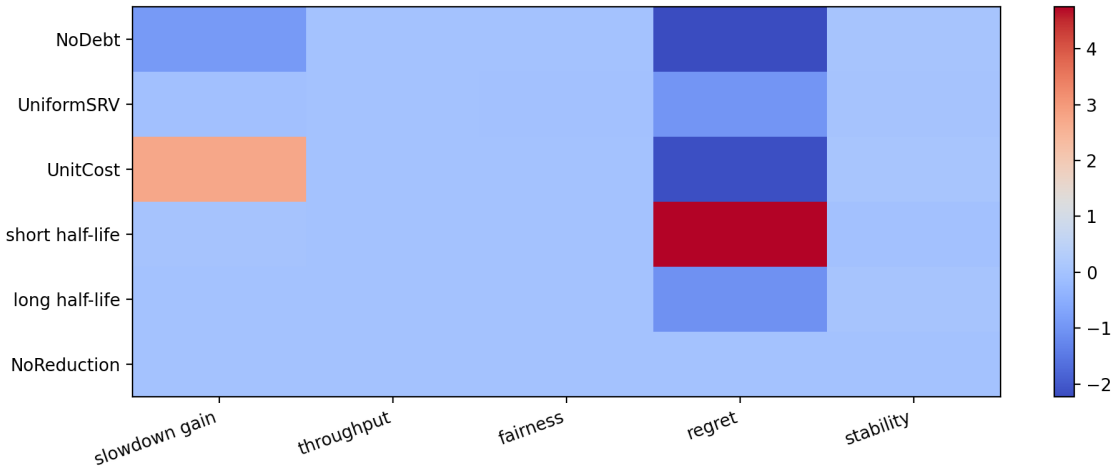

Figure 4: Ablation summary heatmap produced from matched run pairs against full ShareArb. Rows correspond to NoDebt ($n = 36$ tuples), UniformSRV ($n = 12$), UnitCost ($n = 36$), short half-life ($n = 18$), long half-life ($n = 18$), and NoReduction ($n = 18$). Negative slowdown and regret are better; positive throughput and fairness are better.

is inconclusive in the paired comparison against pCache-Account, the debt mechanism is not isolated cleanly by the ablations, the disjoint throughput tolerance check is not met, and the study is limited to shortened replay traces because the live proxy was infeasible.

The practical implication is that responsibility-aware replay arbitration is promising, but the decisive ingredient may be better causal attribution of eviction harm rather than debt accumulation alone. Future work should test that hypothesis in a kernel-backed implementation with longer traces, richer overlap structure, and live observability of refault attribution.

References

Xuechun Cao, Shaurya Patel, Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. Fetchbpf: Customizable prefetching policies in linux with ebpf. In *USENIX Annual Technical Conference (USENIX ATC)*, 2024. URL `https://www.usenix.org/system/files/atc24-cao.pdf`.

Hyoseung Kim and Ragunathan Rajkumar. Shared-page management for improving the temporal isolation of memory reservations in resource kernels. In *IEEE 18th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2012. URL `https://intra.ece.ucr.edu/~hyoseung/pdf/rtcsa12-hyos.pdf`.

Seongbeom Kim, Dhruba Chandra, and Yan Solihin. Fair cache sharing and partitioning in a chip multiprocessor architecture. In *13th International Conference on Parallel Architecture and Compilation Techniques (PACT)*, 2004. URL `https://people.cs.pitt.edu/~melhem/courses/3410p/papers_p/fair.pdf`.

Mayuresh Kunjir, Brandon Fain, Kamesh Munagala, and Shivnath Babu. Robus: Fair cache allocation for data-parallel workloads. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 219–234, 2017. doi: 10.1145/3035918.3064018. URL `https://doi.org/10.1145/3035918.3064018`.

Dusol Lee, Inhyuk Choi, Chanyoung Lee, Sungjin Lee, and Jihong Kim. P2cache: An application-directed page cache for improving performance of data-intensive applications. In *HotStorage*, 2023. doi: 10.1145/3599691.3603408. URL `https://www.hotstorage.org/2023/papers/hotstorage23-final51.pdf`.

External-validity replay check

condition	replay_winner	sign_agreement	slowdown_delta	throughput_delta
External-Raw-2T::tight::seed11	ShareArb	True	0.057524904202949756	-6.224304056863875e-07
External-Raw-2T::medium::seed11	ShareArb	True	0.07550143676635912	-0.00016433574985922836
External-Bursty-2T::tight::seed11	pCache-Account	False	-0.05392959769026362	-1.5593884306843407e-05
External-Bursty-2T::medium::seed11	ShareArb	True	0.15100287353273956	-2.620403401829008e-05
External-Raw-2T::tight::seed23	ShareArb	True	0.057524904202949756	-6.224304056863875e-07
External-Raw-2T::medium::seed23	ShareArb	True	0.07550143676635912	-0.00016433574985922836
External-Bursty-2T::tight::seed23	pCache-Account	False	-0.05392959769026362	-1.5593884306843407e-05
External-Bursty-2T::medium::seed23	ShareArb	True	0.15100287353273956	-2.620403401829008e-05
External-Raw-2T::tight::seed37	ShareArb	True	0.057524904202949756	-6.224304056863875e-07
External-Raw-2T::medium::seed37	ShareArb	True	0.07550143676635912	-0.00016433574985922836
External-Bursty-2T::tight::seed37	pCache-Account	False	-0.05392959769026362	-1.5593884306843407e-05
External-Bursty-2T::medium::seed37	ShareArb	True	0.15100287353273956	-2.620403401829008e-05

Figure 5: Auxiliary replay-side perturbation check rendered from the `external_validity` rows in `results.json`. The table covers 12 raw and burst-perturbed SQLite conditions from `exp/external_validation/`; ShareArb wins 9 of 12 on worst-tenant slowdown. Despite the filename, this figure is *not* a live-validation result and does not substitute for the skipped live step.

Jonggyu Park, Kwonje Oh, and Young Ik Eom. Towards application-level i/o proportionality with a weight-aware page cache management. In *IEEE 36th Symposium on Mass Storage Systems and Technologies (MSST)*, 2020. URL `https://pure.skku.edu/en/publications/towards-application-level-io-proportionality-with-a-weight-aware-`.

Moinuddin K. Qureshi and Yale N. Patt. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *39th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2006. URL `https://www.cs.utexas.edu/~patt/10s.382N/handouts/utility-based-partitioning.pdf`.

Prateek Sharma, Purushottam Kulkarni, and Prashant J. Shenoy. Per-vm page cache partitioning for cloud computing platforms. In *International Conference on Communication Systems and Networks (COMSNETS)*, pages 1–8, 2016. doi: 10.1109/COMSNETS.2016.7439971. URL `https://people.cs.umass.edu/~prateeks/papers/PageCache_comsnets.pdf`.

Ding Tian, Hong Jiang, Khyati Agrawal, and Kenli Li. Characterizing storage workloads with counter stacks. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 15)*, 2015. URL `https://www.usenix.org/conference/osdi15/technical-sessions/presentation/tian`.

Carl Waldspurger, Trausti Saemundsson, Irfan Ahmad, and Nohhyun Park. Cache modeling and optimization using miniature simulations. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 487–498, 2017. URL `https://www.usenix.org/conference/atc17/technical-sessions/presentation/waldspurger`.

stage	observed_sec	note	gpu
trace_capture	observed	see calibration + traces	CPU-only
primary_replay	10136.293701608316	sum of per-run runtime_sec	no GPU
ablations	6533.350309417583	sum of per-run runtime_sec	no GPU
oracle	282.48516751721036	sum of per-run runtime_sec	no GPU

Figure 6: Runtime accounting figure emitted by `exp/shared/analysis.py`. The figure tabulates only the primary replay, ablation, and oracle stages, which sum to 16,952.13 s (4.71 h) of serial CPU time. The separate auxiliary replay perturbation checks reported in text add 1,055.03 s, bringing the replay total to 18,007.16 s (5.00 h).

Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient MRC construction with SHARDS. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 95–110, 2015. URL `https://www.usenix.org/conference/fast15/technical-sessions/presentation/waldspurger`.

Kun Wang, Song Wu, Shengbang Li, Zhuo Huang, Hao Fan, Chen Yu, and Hai Jin. Precise control of page cache for containers. *Frontiers of Computer Science*, 18(2):182102, 2024. doi: 10.1007/s11704-022-2455-0. URL `https://journal.hep.com.cn/fcs/EN/10.1007/s11704-022-2455-0`.

Anil Yelam, Kan Wu, Zhiyuan Guo, Suli Yang, Rajath Shashidhara, Wei Xu, Stanko Novakovic, Alex C. Snoeren, and Kimberly Keeton. Pageflex: Flexible and efficient user-space delegation of linux paging policies with ebpf. In *USENIX Annual Technical Conference (USENIX ATC)*, 2025. URL `https://www.usenix.org/system/files/atc25-yelam.pdf`.

Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. cache_ext: Customizing the page cache with ebpf. In *Symposium on Operating Systems Principles (SOSP)*, 2025. URL `https://www.asafcidon.com/uploads/5/9/7/0/59701649/cache_ext.pdf`.

A Reproducibility Appendix

A.1 Directory-to-paper mapping

A.2 Fixed seeds, lengths, and metric formulas

All reported stochastic components use seeds $\{11, 23, 37\}$; the held-out pilot seed is 5 and is not reported. The executed replay regime uses 20,000 synthetic references per tenant and 15,000 SQLite references per tenant. On seed 11, this yields event counts of 40,000 for OverlapShift-2T, ScanVsLoop-2T, and DisjointPhase-2T, 30,000 for SQLiteTraceMix-2T, and 45,000 for SQLiteTraceMix-3T.

The exact metric formulas are implemented in `exp/shared/replay.py`:

- worst-tenant slowdown: $\max_i \left(\overline{\text{latency}}_i / \overline{\text{latency}}_i^{\text{isolated}}\right)$;
- aggregate throughput proxy: total references divided by total replay latency;
- Jain fairness: Jain's index over inverse slowdowns;

Table 3: Artifact-to-paper mapping.

Artifact path	Role in paper
`exp/primary/`	Main replay matrix in Section 4
`exp/ablations/`	Ablations in Section 4
`exp/oracle/`	OracleOverlap reference in Section 4
`exp/external_validation/`	Auxiliary replay perturbation check
`exp/live_validation/`	Skipped live-proxy step; claims scoped away
`exp/smoke/`	Runtime-gating artifact for shortened traces
`traces/`	Synthetic and SQLite-derived replay inputs
`calibration/manifest.json`	Budgets, miss costs, isolated baselines
`figures/`	Generated figures included in the paper
`results.json`	Aggregated statistics and claim assessment

- shared regret per 10,000 references: `shared_regret_raw` $\times 10{,}000/|\text{trace}|$;
- controller changes per 10,000 references: action-change count $\times 10{,}000/|\text{trace}|$;
- debt-harm alignment: correlation between charged debt in epoch t and realized downstream harm in epoch $t+1$ over tenant-epoch pairs.

A.3 Figure and table provenance

Running `python3 -m exp.analysis.run` regenerates `results.json` and the figure files in `figures/`. The plotting logic is defined in `exp/shared/analysis.py`. Specifically:

- Figure 1 uses medium-budget rows from the primary results and plots seed means with bootstrap error bars.
- Figure 2 plots mean worst-tenant slowdown versus budget for the selected methods.
- Figure 4 merges each ablation against ShareArb on matched workload-budget-seed tuples and plots mean deltas.
- Figure 3 scatters `debt` versus `realized_harm_next` from all ShareArb `epoch_metrics.jsonl` rows.
- Figure 5 renders the auxiliary replay perturbation table from `results.json`.
- Figure 6 is generated from summed `runtime_sec` values for the primary, ablation, and oracle stages only.

Table 1 was transcribed from the medium-budget aggregates in `results.json`. Table 2 uses medium-budget overlap-heavy means computed from the same aggregated results.

A.4 Smoke and skipped live-validation artifacts

Table 4: Concise accounting of smoke and live-validation artifacts excluded from the main empirical narrative.

Artifact	Status	Reason for exclusion from main results
`exp/smoke/`	5 runs executed	Used only to trigger the runtime fallback that shortened traces from 80,000 to 20,000 synthetic references per tenant and from 60,000 to 15,000 SQLite references per tenant.
`FULL_LENGTH_PROBE.md`	partial probe only	Records infeasible full-length medium-budget runtimes, including 553.56 s for PrivateOnly-Utility and 544.33 s for pCache-Account on OverlapShift-2T seed 11.
`exp/live_validation/`	skipped	Writable cgroup `memory.high` controls, cache resets, and residency sampling hooks were unavailable without elevated privileges.
`external_validation`	auxiliary replay check	Retained as a perturbation check on captured SQLite streams and reported separately; it is not a substitute for the skipped live step.